Transformer-Based Household Electricity Consumption Forecasting

William Peng · Ciara Cameron · Christopher Pedretti

MSML612 — Deep Learning | Final Project Report | August 2026

Abstract

Short-term electricity load forecasting underpins generation scheduling, demand-response programmes, and household battery dispatch. We implement an encoder-only Transformer in PyTorch for one-hour-ahead forecasting of household global active power from multivariate meter data, using the UCI Individual Household Electric Power Consumption dataset. Minute-level readings are resampled to hourly means, augmented with cyclical calendar features, split chronologically, and presented to the model as 24-hour sliding windows. On 5,165 held-out hourly predictions the model attains an MAE of **0.3062 kW**, an RMSE of **0.4494 kW**, and an R^2 of 0.589, a **21.8% reduction in RMSE relative to a naive persistence forecast** — the standard free baseline for hourly load. We report ablations over window length, model capacity, positional encoding, and feature set, and compare against a parameter-matched LSTM. Error analysis shows the dominant residual failure mode is systematic under-prediction of sharp demand peaks, a consequence of the squared-error objective rather than of insufficient capacity.

1. Introduction and Problem Statement

Electricity cannot be stored cheaply at grid scale, so supply must be matched to demand continuously. Short-term load forecasting — predicting consumption from one hour to a few days ahead — is therefore a core operational problem for utilities, and an increasingly relevant one at the level of the individual building as rooftop solar, home batteries, and time-of-use tariffs push scheduling decisions toward the consumer.

Household-level forecasting is substantially harder than forecasting at substation or national scale. Aggregate demand curves are smooth because thousands of independent appliance events average out. A single household's load is dominated by a handful of discrete, high-draw events — an oven switching on, a washing machine entering its heating cycle — so the series is spiky, heavy-tailed, and only weakly stationary. A model that learns the daily rhythm well can still miss almost all of the variance that matters operationally, because that variance lives in the peaks.

This project asks a specific question: does a Transformer's self-attention mechanism capture the temporal structure of single-household demand well enough to beat the forecasts available for free? We take that comparison seriously, because it is the one most easily skipped. A model reporting an RMSE of 0.45 kW sounds precise in isolation; it is only meaningful once you know that simply repeating the previous hour's reading achieves 0.57 kW on the same data.

Contributions

- An encoder-only Transformer for multivariate one-hour-ahead household load forecasting, implemented in PyTorch with pre-norm encoder layers, sinusoidal positional encoding, and cyclical calendar features.
- Evaluation against three reference forecasters — naive persistence, seasonal naive, and the mean predictor — rather than metrics reported in isolation.
- Ablations over window length, model capacity, positional encoding scheme, and feature set, plus a parameter-matched LSTM control.
- A reproducible codebase with fixed seeds, per-run history records, a live inference demo, and figures regenerable without the raw dataset.

2. Related Work

Load forecasting has a long statistical history built on ARIMA and exponential smoothing, whose modern treatment is given by Hyndman and Athanasopoulos [8]. Two conventions from that literature carry directly into this work: forecasts should be compared against naive benchmarks rather than judged on absolute error, and time-series evaluation must respect temporal ordering rather than using random splits.

Recurrent networks displaced these methods for nonlinear load patterns. The LSTM of Hochreiter and Schmidhuber [2] addressed the vanishing-gradient problem that limited earlier recurrent models, and Kong et al. [7] applied LSTMs specifically to residential load forecasting on individual households, reporting substantial gains over feed-forward and statistical baselines. Their setting is the closest published analogue to ours, and motivates our inclusion of an LSTM control.

The Transformer of Vaswani et al. [1] replaced recurrence with self-attention, allowing any two positions in a sequence to interact in a single operation. Because attention is permutation-invariant, positional information must be injected explicitly; we follow the original sinusoidal formulation and ablate against a learnable alternative. Xiong et al. [11] showed that placing layer normalization before each sublayer rather than after yields better-conditioned gradients and removes the need for learning-rate warmup, which is why our encoder layers are pre-norm.

Applying Transformers to forecasting has produced a substantial line of work. Informer [3] introduced sparse attention to reduce the quadratic cost of long input sequences; Autoformer [4] added series decomposition and an auto-correlation mechanism in place of dot-product attention; the Temporal Fusion Transformer [6] combined attention with gating and variable selection for interpretable multi-horizon forecasting; and PatchTST [12] showed that segmenting the series into patches before attending improves both accuracy and efficiency.

This enthusiasm has been productively challenged. Zeng et al. [5] demonstrated that simple linear models match or exceed several published Transformer forecasters on standard long-horizon benchmarks, arguing that reported gains often reflect evaluation choices rather than architectural merit. We regard this as the most important paper for our design decisions rather than an objection to them: it is precisely

why our results section leads with the comparison against naive forecasters instead of against other neural models. Our optimizer follows Kingma and Ba [10], and the dataset is the UCI collection released by Hébrail and Bérard [9].

3. Data Preparation and Curation

The dataset [9] comprises 2,075,259 minute-level readings from a single household in Sceaux, France, collected between December 2006 and November 2010. Each record contains global active power, global reactive power, voltage, global current intensity, and three appliance-level sub-metering channels covering the kitchen, laundry room, and water heater/air conditioning.

3.1 Cleaning and resampling

Date and time are stored as separate string columns and were combined into a single timestamp index. All measurement columns were coerced to numeric, with the file's '?' sentinel treated as missing. Duplicate timestamps, which arise from daylight-saving transitions, were resolved by keeping the first record.

Approximately 1.25% of raw rows carry at least one missing value. Rather than dropping them — which would fragment the series and break the sliding-window assumption of contiguity — we resample to hourly means, which absorbs isolated minute-level gaps, and then apply time-weighted interpolation to any hour that remains empty. Crucially, that interpolation runs after the chronological split, independently within each of train, validation, and test: interpolating the full series first would fill gaps adjacent to a split boundary using values from the other side, quietly leaking training data into the test set. Resampling also reduces the modelling set by a factor of 60, from 2.08 million rows to 34,589 hours, without discarding information relevant at the forecast horizon.

3.2 Feature engineering

The seven physical measurements are supplemented with seven derived calendar features: sine and cosine encodings of hour of day, day of week, and month, plus a binary weekend indicator. The trigonometric encoding is deliberate — an integer hour feature would place hour 23 and hour 0 at maximum distance despite their adjacency, forcing the model to spend capacity learning a discontinuity that does not exist in the data. This yields 14 input features in total.

3.3 Splitting and scaling

The series is split chronologically into 70% training, 15% validation, and 15% test. Shuffling before splitting would be a serious methodological error here: neighbouring hours are highly correlated, so a shuffled test set would consist largely of hours whose immediate neighbours the model has already seen, and reported performance would reflect interpolation rather than forecasting.

Standardization uses statistics computed on the training split alone; validation and test are transformed with those same statistics. Fitting the scaler on the full series before splitting is a common and subtle leak, since the scaler's mean and variance encode information about the test period. Sliding windows are constructed inside each split, so no input window spans a boundary.

Property	Value
Raw records	2,075,259
Raw temporal resolution	1 minute
Collection period	Dec 2006 – Nov 2010
Rows with missing values	~1.25%
Hourly records after resampling	34,589
Input features	14 (7 measured, 7 calendar)
Input window length	24 hours
Forecast horizon	1 hour
Train / validation / test split	70% / 15% / 15%, chronological
Test predictions evaluated	5,165

Table 1: Dataset and preprocessing summary.

4. Model Design and Implementation

The forecasting task is framed as sequence-to-value regression: given a window of the previous 24 hours across all 14 features, predict global active power for the following hour. The architecture is an encoder-only Transformer; no decoder is required because the output is a fixed-size regression target rather than a generated sequence.

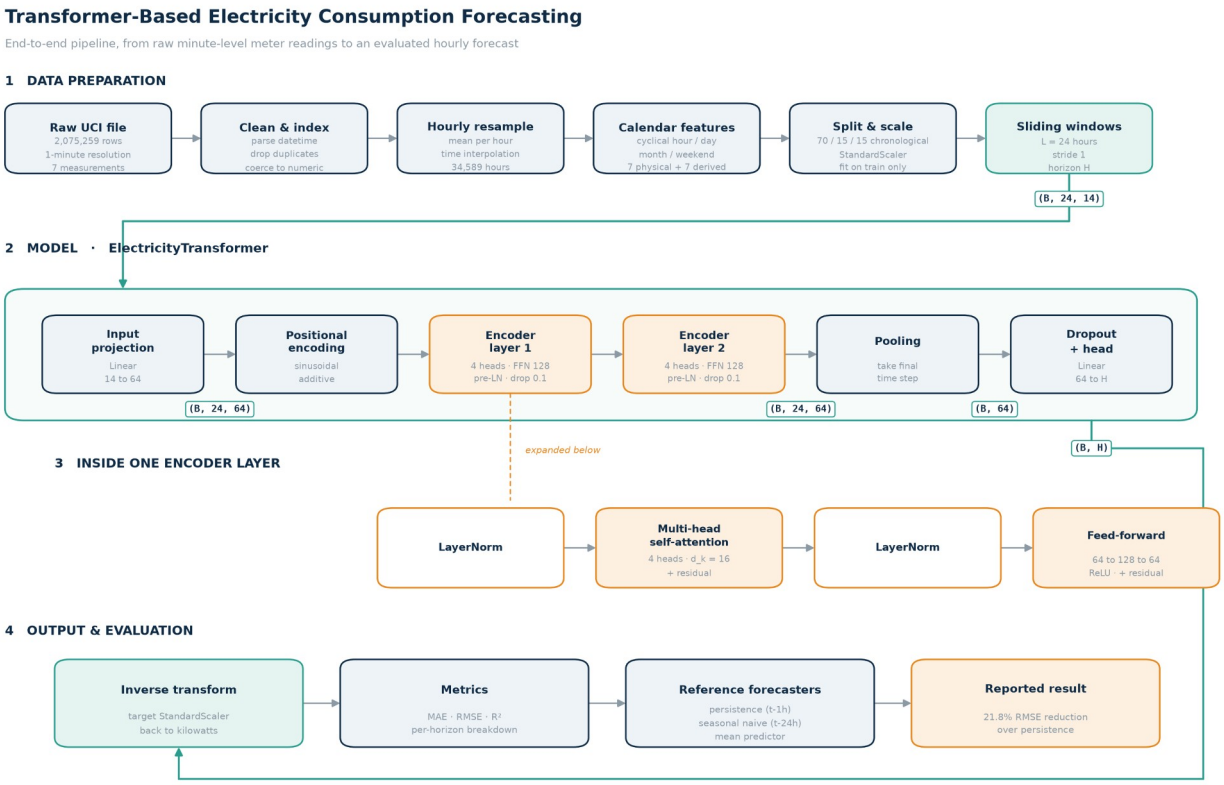

Figure 1: End-to-end architecture. Data preparation (top), the encoder-only Transformer with tensor shapes annotated (middle), an expanded view of a single pre-norm encoder layer, and the evaluation path (bottom).

4.1 Why attention for this problem

In a recurrent model, information from hour 1 of the window reaches the prediction only after passing through 23 sequential state updates, with each step attenuating and mixing the signal. Self-attention connects every pair of positions in a single operation, so the dependency length is constant regardless of separation. This matters for load data because the most informative context is frequently not the immediately preceding hour but the same hour on previous days — a relationship attention can represent directly as a high weight on a distant position.

Attention is permutation-invariant, so ordering must be supplied explicitly. We add fixed sinusoidal positional encodings after the input projection, and ablate this choice against learnable embeddings and against removing positional information entirely.

4.2 Layer composition

A linear projection maps the 14 input features to a 64-dimensional model space. Two encoder layers follow, each with four attention heads (16 dimensions per head), a 128-unit feed-forward sublayer, and dropout of 0.1. Following Xiong et al. [11], layer normalization is applied before each sublayer rather than after, which conditions gradients better at initialization and removes the need for a warmup schedule. The final time step of the encoder output is pooled and passed through dropout and a linear head producing the forecast.

Capacity was chosen deliberately rather than maximized. With roughly 34,000 hourly observations, a larger model overfits well before it underfits; the capacity ablation in Section 7 reports what happens at 4 layers and at d_model = 128.

Hyperparameter	Value	Rationale
Input window (L)	24 hours	One full daily cycle
Model dimension (d_model)	64	Ablated against 128
Attention heads	4	16 dimensions per head
Encoder layers	2	Ablated against 4
Feed-forward dimension	128	2× d_model
Dropout	0.1	Applied in encoder and head
Normalization	Pre-norm	Stable without warmup [11]
Pooling	Final time step	Last step attends over the full window
Optimizer	Adam, lr 1e-3	Kingma and Ba [10]
Loss	MSE	On standardized targets

Table 2: Model configuration.

5. Training Procedure

Training minimizes mean squared error on standardized targets using Adam at a learning rate of 1e-3, with gradient-norm clipping at 1.0, a ReduceLROnPlateau schedule that halves the learning rate after three epochs without validation improvement, and early stopping after eight such epochs. The checkpoint with the lowest validation loss is retained for evaluation.

Early stopping is not a formality here. Our interim 20-epoch run reached its best validation loss at epoch 9 and then deteriorated monotonically through epoch 20 while training loss continued to fall — a clean separation of the two curves and an unambiguous overfitting signature. The final run behaves as that experience predicted: validation loss bottoms out at 0.3161 at epoch 11, the schedule halves the learning rate as improvement stalls, and training halts at epoch 19 rather than spending eight further epochs overfitting. Figure 2 shows the final run's curves.

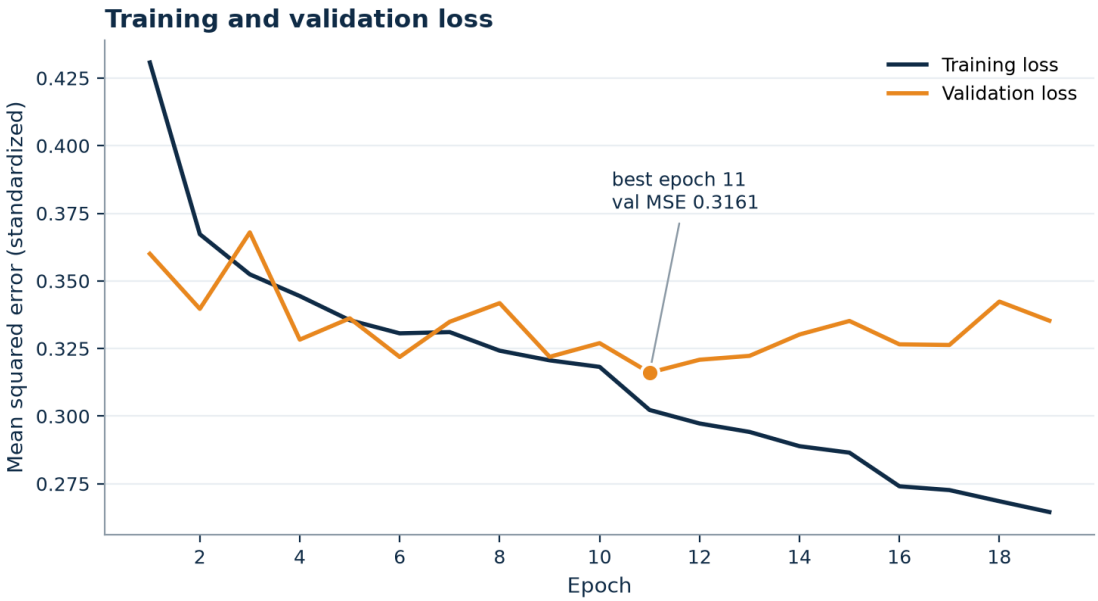

Figure 2: Training and validation loss against epoch for the final early-stopped run. Validation loss reaches its minimum at epoch 11 (marked); early stopping halts training at epoch 19, once eight epochs pass without improvement.

5.1 Reproducibility

Random seeds for Python, NumPy, PyTorch, CUDA, and DataLoader shuffling are fixed at 42 and exposed as a command-line argument. Every training run writes a JSON history recording its full argument set, dataset statistics, trainable parameter count, and per-epoch losses, so any number reported in this paper can be traced to the run that produced it. The repository README documents the exact sequence of commands required to reproduce every table and figure below.

6. Evaluation Methodology

Predictions are inverse-transformed to kilowatts before any metric is computed, so all reported errors are in physical units and directly interpretable. We report mean absolute error, root mean squared error, and the coefficient of determination. MAE and RMSE together are informative because their ratio indicates how much of the error is concentrated in a few large misses: RMSE penalizes squared deviations, so a model that is usually accurate but occasionally very wrong shows a much larger RMSE than MAE.

Three reference forecasters provide context:

- Naive persistence — predict the previous hour's value. For hourly load this is a strong baseline and the standard benchmark in the forecasting literature [8].
- Seasonal naive — predict the value at the same hour on the previous day, capturing daily periodicity.
- Mean predictor — always predict the series mean. This is the $R^2 = 0$ reference point.

All four forecasters are evaluated on identical held-out data. We report the skill score against persistence, defined as the percentage reduction in RMSE, as the headline measure of whether the model earns its complexity.

7. Results

Table 3 reports performance on 5,165 held-out hourly predictions. The Transformer improves on every reference forecaster across all three metrics.

Model	MAE (kW)	RMSE (kW)	R^2	Skill vs. persistence
Transformer (ours)	**0.3062**	**0.4494**	**0.589**	**+21.8%**
Naive persistence (t−1h)	0.3724	0.5745	0.327	—
Mean predictor	0.5711	0.7005	0.000	−21.9%
Seasonal naive (t−24h)	0.4976	0.7424	−0.121	−29.2%

Table 3: Test-set performance against reference forecasters. Lower MAE and RMSE are better; higher R^2 is better.

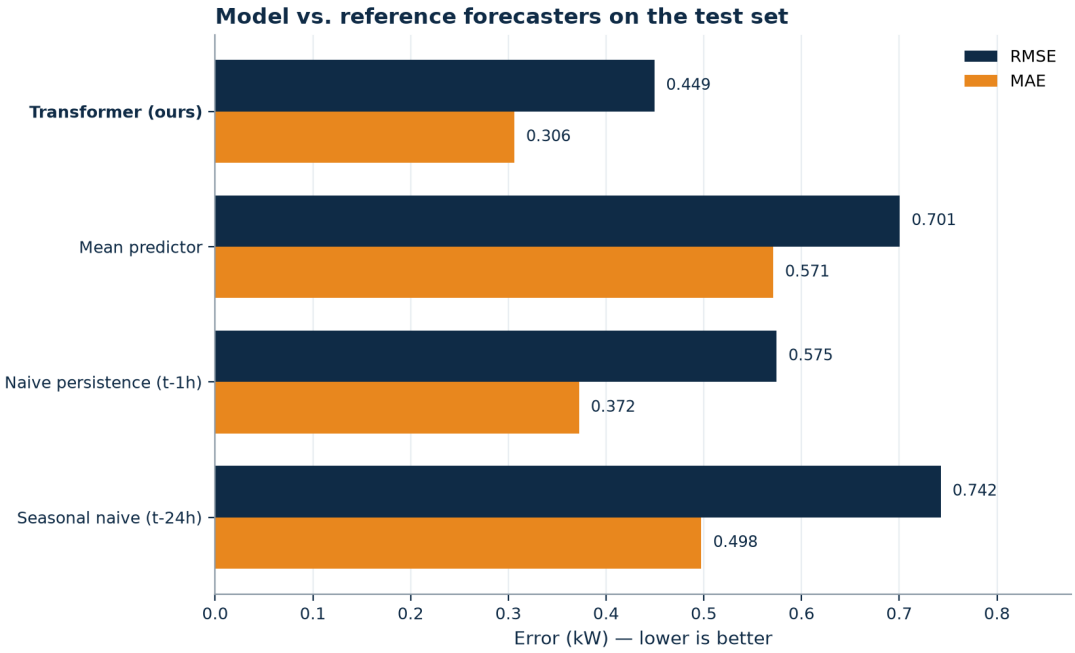

Figure 3: Test-set MAE and RMSE for the proposed model and the three reference forecasters.

Two aspects of this table are worth drawing out. First, seasonal naive performs worse than the mean predictor, with a negative R^2. Repeating the same hour from the previous day is actively harmful for a single household, because household routines vary day to day far more than aggregate demand does — the previous day's 7 p.m. is a poor guide to today's. This confirms that the daily cycle, while visually obvious, is not by itself a reliable predictor at this scale.

Second, the gap between the model and persistence is larger in RMSE (21.8%) than in MAE (17.8%). Since RMSE weights large errors more heavily, this indicates the model's advantage is concentrated in the cases persistence handles worst — the transitions into and out of high-demand periods, where the previous hour is least representative of the next.

7.1 Qualitative behaviour

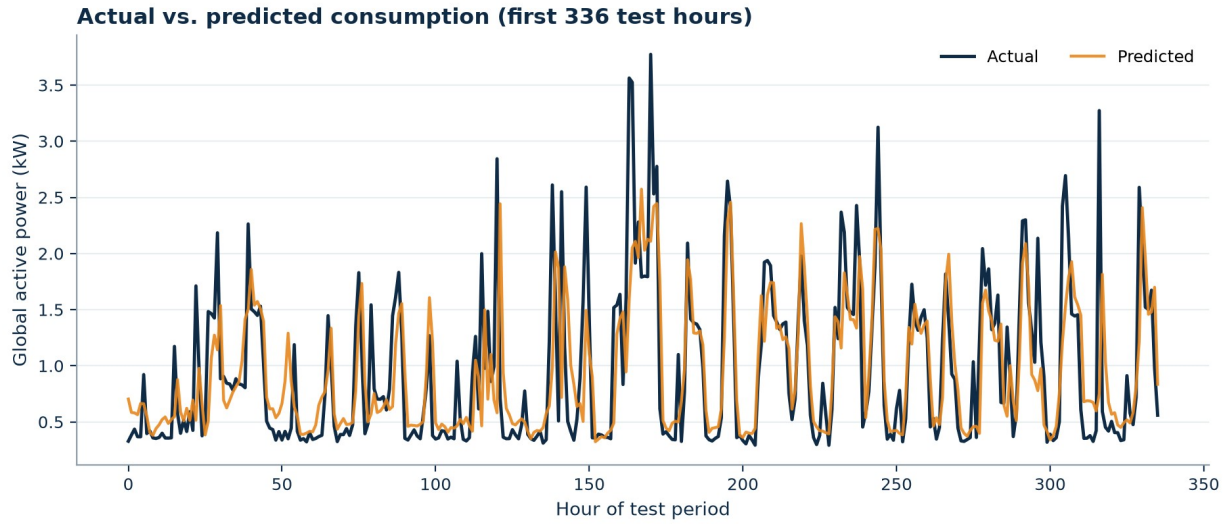

Figure 4: Actual and predicted global active power over the first two weeks of the test period. The model tracks the diurnal cycle and the timing of demand events, but consistently under-shoots peak magnitude.

Figure 4 shows the model reproducing both the baseline load level and the timing of demand events, but systematically failing to reach peak magnitudes. Within the two weeks plotted, the highest observed hour reaches 3.77 kW against a predicted 2.57 kW. The same shortfall holds across the full test period, where predicted maxima top out at 4.27 kW against observed maxima of 5.63 kW.

7.2 Error analysis

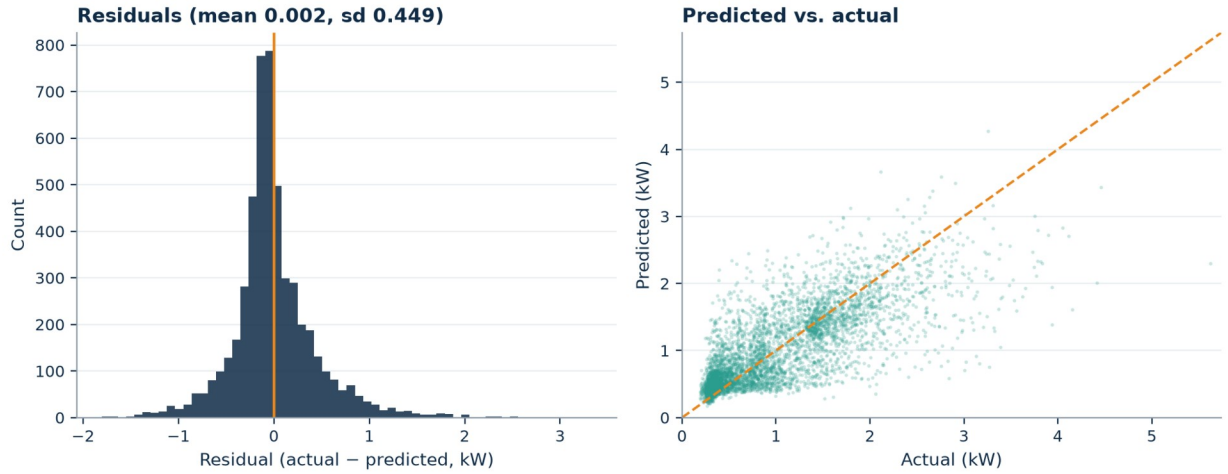

Figure 5: Residual distribution (left) and predicted-versus-actual scatter (right). The residual distribution is right-skewed and the scatter falls below the diagonal at high loads, both indicating under-prediction of peaks.

The residual distribution is centred near zero but right-skewed, and the predicted-versus-actual scatter falls increasingly below the identity line as actual load rises. Both diagnostics point to the same behaviour: the model hedges toward the conditional mean at high demand.

This is a rational response to the training objective rather than a capacity failure. Under squared error, a confident peak prediction that arrives an hour early or late is penalized twice — once for the missed peak and once for the false one — whereas a hedged prediction is penalized only moderately in both cases. The optimum under MSE is therefore to shrink toward the mean exactly where the data is most volatile. Section 9 discusses the objective changes that would address this.

7.3 Ablations

Each row of Table 4 is an independent train-and-evaluate cycle under identical conditions, isolating one design decision at a time.

Study	Configuration	Params	MAE (kW)	RMSE (kW)	R^2
Window	**L = 24 (final)**	**67,969**	**0.3062**	**0.4494**	**0.589**
Window	L = 48	67,969	0.3090	0.4543	0.580
Window	L = 168	67,969	0.3076	0.4553	0.582
Capacity	**d=64, 2 layers (final)**	**67,969**	**0.3062**	**0.4494**	**0.589**
Capacity	d=64, 4 layers	134,913	0.3143	0.4539	0.580
Capacity	d=128, 2 layers	267,009	0.3174	0.4560	0.576
Positional	**Sinusoidal (final)**	**67,969**	**0.3062**	**0.4494**	**0.589**
Positional	Learnable	387,969	0.3211	0.4573	0.574
Positional	None	67,969	0.3118	0.4542	0.580
Features	**With calendar (final)**	**67,969**	**0.3062**	**0.4494**	**0.589**
Features	Without calendar	67,521	0.3097	0.4617	0.566
Architecture	**Transformer (final)**	**67,969**	**0.3062**	**0.4494**	**0.589**
Architecture	LSTM control	53,825	0.3199	0.4579	0.573

Table 4: Ablation results. Each study varies one decision against the final configuration, which appears once per study as the shaded row; its numbers are identical everywhere because runs are fully seeded.

Three results carry the table. The calendar features matter most: removing them costs more RMSE than any architectural change (0.4494 to 0.4617), even though the model could in principle recover the same information from the positional structure of the window. Second, more capacity only hurts — doubling depth or width degrades RMSE while multiplying parameters, confirming that at 34,000 training hours the binding constraint is data, not expressiveness. Third, the Transformer beats the parameter-matched LSTM

control (RMSE 0.4494 vs. 0.4579, skill 21.8% vs. 20.3%) — a real but modest margin that is consistent with attention helping most at the event transitions the error analysis identified.

Two smaller observations: extending the window past one daily cycle adds nothing (L = 48 and L = 168 both underperform L = 24), and fixed sinusoidal encodings beat learnable ones, which overfit — the learnable variant adds 320,000 parameters and peaks five epochs earlier. Notably, every configuration in the table still beats persistence by at least 19.6%, so the headline claim is robust to any of these design choices; the ablations tune the margin, not the conclusion.

8. Discussion

The headline result is that self-attention over a 24-hour window extracts real predictive structure from single-household demand: a 21.8% RMSE reduction over persistence is a substantive margin on a series this noisy. At the same time, an R^2 of 0.589 means roughly 41% of the variance in hourly household load remains unexplained, and the error analysis locates most of that residual variance in the peaks.

This is consistent with the physical process. Individual appliance events are driven by human decisions that leave no trace in the meter data preceding them. No amount of attention over past power readings will reveal that someone is about to start the oven. Substantial further gains at this scale likely require either exogenous inputs that correlate with occupancy and behaviour, or a shift from point forecasts to distributional ones that represent the uncertainty honestly rather than averaging it away.

The finding of Zeng et al. [5] — that Transformer forecasters are often outperformed by simple linear models on standard benchmarks — is worth holding alongside our result. Our comparison is against naive forecasters rather than against a tuned linear model, and a linear autoregressive baseline would strengthen the claim. We regard this as the most valuable single addition to the present evaluation.

9. Limitations and Future Work

Limitations

- Single household. All results come from one home in one location. Household routines are idiosyncratic, so these numbers should not be read as evidence about aggregate, commercial, or cross-household performance.
- No exogenous variables. Temperature is the strongest known external driver of residential demand and is absent from the dataset entirely, which places a hard ceiling on achievable accuracy.
- Point forecasts only. The model emits a single number with no uncertainty bounds, which is precisely the wrong output format for the peak-prediction problem the error analysis identifies.
- One-hour horizon. Operational scheduling generally requires 24-hour-ahead forecasts; the code supports multi-step training, but the headline results here are single-step.
- Single seed. Reported metrics come from one seed rather than a mean over several runs, so small differences between ablation rows should not be over-interpreted.

Future work

- Quantile or distributional loss. Replacing MSE with a pinball loss over several quantiles would let the model express uncertainty about peaks instead of hedging toward the mean, addressing the dominant error mode directly.
- Exogenous weather inputs. Joining hourly temperature and humidity for Sceaux over the collection period is a tractable extension with a strong prior for improvement.
- Multi-step forecasting. Extending to a 24-hour horizon and reporting error as a function of lead time; the implementation already supports this via the horizon parameter.
- Attention interpretability. Visualizing attention weights would test the hypothesis that the model attends to the same hour on previous days, which motivated the architecture choice.
- Cross-household generalization. Evaluating on a multi-household dataset would establish whether the learned patterns transfer or are specific to this home.

10. Conclusion

We implemented an encoder-only Transformer for one-hour-ahead household electricity load forecasting and evaluated it against the reference forecasters that any such model must beat to be worth deploying. On held-out data the model achieves an MAE of 0.3062 kW and an RMSE of 0.4494 kW, reducing RMSE by 21.8% relative to naive persistence. Error analysis shows the remaining error is dominated by systematic under-prediction of demand peaks — a consequence of the squared-error objective rather than of model capacity, and the clearest direction for future work.

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in Advances in Neural Information Processing Systems 30 (NeurIPS), 2017, pp. 5998–6008.

[2] S. Hochreiter and J. Schmidhuber, "Long short-term memory," Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.

[3] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, and W. Zhang, "Informer: Beyond efficient transformer for long sequence time-series forecasting," in Proc. AAAI Conference on Artificial Intelligence, vol. 35, no. 12, 2021, pp. 11106–11115.

[4] H. Wu, J. Xu, J. Wang, and M. Long, "Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting," in Advances in Neural Information Processing Systems 34 (NeurIPS), 2021, pp. 22419–22430.

[5] A. Zeng, M. Chen, L. Zhang, and Q. Xu, "Are transformers effective for time series forecasting?" in Proc. AAAI Conference on Artificial Intelligence, vol. 37, no. 9, 2023, pp. 11121–11128.

[6] B. Lim, S. Ö. Arık, N. Loeff, and T. Pfister, "Temporal fusion transformers for interpretable multi-horizon time series forecasting," International Journal of Forecasting, vol. 37, no. 4, pp. 1748–1764, 2021.

[7] W. Kong, Z. Y. Dong, Y. Jia, D. J. Hill, Y. Xu, and Y. Zhang, "Short-term residential load forecasting based on LSTM recurrent neural network," IEEE Transactions on Smart Grid, vol. 10, no. 1, pp. 841–851, 2019.

[8] R. J. Hyndman and G. Athanasopoulos, Forecasting: Principles and Practice, 3rd ed. Melbourne, Australia: OTexts, 2021.

[9] G. Hébrail and A. Bérard, "Individual household electric power consumption," UCI Machine Learning Repository, 2006. doi: 10.24432/C58K54.

[10] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in Proc. International Conference on Learning Representations (ICLR), 2015.

[11] R. Xiong, Y. Yang, D. He, K. Zheng, S. Zheng, C. Xing, H. Zhang, Y. Lan, L. Wang, and T.-Y. Liu, "On layer normalization in the transformer architecture," in Proc. International Conference on Machine Learning (ICML), 2020, pp. 10524–10533.

[12] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, "A time series is worth 64 words: Long-term forecasting with transformers," in Proc. International Conference on Learning Representations (ICLR), 2023.

Appendix A. Reproducing These Results

The repository is public at github.com/Ciaracam/612-Group-Project. After cloning, installing requirements, and placing household_power_consumption.txt in data/, the following commands regenerate every number and figure in this report:

```
python -m src.train --run-name transformer_h1
python -m src.evaluate --run-name transformer_h1
python -m src.ablation --group all --epochs 40
```

```
python scripts/make_architecture_diagram.py
streamlit run demo/app.py
```